01_Dateiformate_Pandas

September 23, 2026

[1]:
```
# Installation falls nötig
!python -m pip install --quiet pyarrow fastparquet altair
```

1 Voranalyse: Vergleich von Dateiformaten mit Pandas

1.1 Ziel der Untersuchung

In dieser Voranalyse wird untersucht, wie sich verschiedene Dateiformate beim Speichern und Einlesen eines größeren Datensatzes verhalten.

Verglichen werden insbesondere:

- Dateigröße,
- Schreibdauer,
- Lesedauer,
- benötigter Speicherplatz und
- praktische Eignung für die weitere Analyse.

Als Ausgangsbasis für diesen Performance-Test dient ein originaler Tabellen-Datensatz, der komprimiert im **ZIP-Format** vorliegt. In dem Format ist die Datei leicht über **100 MB** groß.

Aber als pandas DataFrame wird fast **1 GB** benötigen.

Die Untersuchung stellt zunächst einen Vergleich der verfügbaren Dateiformate mit Pandas dar. Die Ergebnisse bilden eine Grundlage für die spätere Auswahl geeigneter Dateiformate im Hauptprojekt.

> Hinweis: Die gemessenen Zeiten können je nach Systemauslastung und Ausführungsbedingungen geringfügig variieren.

[2]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import altair as alt
import os, time, bz2
import pandas as pd
import fastparquet
from pathlib import Path
from project_config import (
    ORIGINAL_DS_ZIP_PATH,
    DATEIFORMATE_RESULTS
)
```

[3]:
```
df = pd.read_csv(ORIGINAL_DS_ZIP_PATH, compression="zip")
```

[4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2061357 entries, 0 to 2061356
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   ID                     str
 1   line                   str
 2   path                   str
 3   eva_nr                 int64
 4   category               int64
 5   station                str
 6   state                  str
 7   city                   str
 8   zip                    int64
 9   long                   float64
 10  lat                    float64
 11  arrival_plan           str
 12  departure_plan         str
 13  arrival_change         str
 14  departure_change       str
 15  arrival_delay_m        int64
 16  departure_delay_m      int64
 17  info                   str
 18  arrival_delay_check    str
 19  departure_delay_check  str
dtypes: float64(2), int64(5), str(13)
memory usage: 947.4 MB
```

[5]:
```
# Definition der Testvarianten - fastparquet ohne Levels
tests = [
    ("CSV", None, None, None, ".csv", {"index": False}),
    (
        "CSV_BZ2",
        None,
        None,
        None,
        ".csv.bz2",
        {"index": False, "compression": "bz2"},
    ),
    (
        "CSV_ZIP",
        None,
        None,
        None,
```

```
        ".csv.zip",
        {"index": False, "compression": "zip"},
    ),
    # Snappy-Vergleich
    ("Parquet", "pyarrow", "snappy", None, ".parquet", {}),
    ("Parquet", "fastparquet", "snappy", None, ".parquet", {}),
    # ZSTD-Vergleich (PyArrow mit echten Levels, fastparquet nur Standard)
    ("Parquet", "pyarrow", "zstd", 3, ".parquet", {}),
    ("Parquet", "pyarrow", "zstd", 10, ".parquet", {}),
    ("Parquet", "pyarrow", "zstd", 11, ".parquet", {}),
    ("Parquet", "fastparquet", "zstd", None, ".parquet", {}),
    # Brotli-Vergleich
    ("Parquet", "pyarrow", "brotli", 5, ".parquet", {}),
    ("Parquet", "pyarrow", "brotli", 10, ".parquet", {}),
    ("Parquet", "pyarrow", "brotli", 11, ".parquet", {}),
    ("Parquet", "fastparquet", "brotli", None, ".parquet", {}),
]

results = []

for fmt, engine, codec, level, ext, extra in tests:
    # Eindeutiger Dateiname für den Testlauf
    lvl_str = f"_lvl{level}" if level is not None else ""
    codec_str = f"_{codec}" if codec is not None else ""
    engine_str = f"_{engine}" if engine is not None else ""
    fname = f"bench_{fmt}{engine_str}{codec_str}{lvl_str}{ext}"

    kwargs = extra.copy()

    # --- SCHREIBEN ---
    start = time.time()
    try:
        if fmt.startswith("CSV"):
            df.to_csv(fname, **kwargs)
        elif fmt == "Parquet":
            # PyArrow akzeptiert das compression_level Argument direkt flach
            if level is not None and engine == "pyarrow":
                kwargs["compression_level"] = level

            df.to_parquet(
                fname, engine=engine, compression=codec, **kwargs
            )

        write_time = time.time() - start
        size_mb = os.path.getsize(fname) / (1024**2)

        # --- LESEN ---
```

```
        start = time.time()
        if fmt.startswith("CSV"):
            _ = pd.read_csv(fname, compression=kwargs.get("compression", None))
        elif fmt == "Parquet":
            _ = pd.read_parquet(fname, engine=engine)

        read_time = time.time() - start

        # Ergebnisse speichern
        results.append(
            {
                "Format": fmt,
                "Engine": engine if engine else "-",
                "Codec": codec if codec else "-",
                "Level": level if level else "Standard",
                "Größe_MB": round(size_mb, 2),
                "Schreibzeit_s": round(write_time, 2),
                "Lesezeit_s": round(read_time, 2),
            }
        )

    except Exception as e:
        print(f"Fehler bei Kombination {fmt}-{engine}-{codec}: {e}")

    finally:
        # Datei in jedem Fall aufräumen, damit die Festplatte nicht zumüllt
        if os.path.exists(fname):
            os.remove(fname)

# DataFrame erstellen
df_results = pd.DataFrame(results)

# Der Eindeutigkeits-Fix für Ihre Diagramme (keine zusammengefassten Balken␣
 ↪mehr!)
df_results["Variante"] = df_results.apply(
    lambda r: f"{r['Format']} ({r['Engine']}) - {r['Codec']} [{r['Level']}]"
    if r["Codec"] != "-"
    else f"{r['Format']} [{r['Level']}]",
    axis=1,
)

display(df_results)
```

	Format	Engine	Codec	Level	Größe_MB	Schreibzeit_s	\
0	CSV	-	-	Standard	736.48	27.17	
1	CSV_BZ2	-	-	Standard	65.79	90.78	
2	CSV_ZIP	-	-	Standard	101.77	41.70	
3	Parquet	pyarrow	snappy	Standard	127.21	2.85	

```
4   Parquet  fastparquet  snappy  Standard    159.71          6.13
5   Parquet      pyarrow    zstd         3     56.45          3.18
6   Parquet      pyarrow    zstd        10     50.14          9.53
7   Parquet      pyarrow    zstd        11     49.69         10.56
8   Parquet  fastparquet    zstd  Standard     50.15          6.61
9   Parquet      pyarrow  brotli         5     50.98          8.85
10  Parquet      pyarrow  brotli        10     44.52        135.78
11  Parquet      pyarrow  brotli        11     44.07        476.22
12  Parquet  fastparquet  brotli  Standard     26.91       1316.48
```

```
    Lesezeit_s                                  Variante
0        13.16                              CSV [Standard]
1        27.37                          CSV_BZ2 [Standard]
2        13.75                          CSV_ZIP [Standard]
3         0.47      Parquet (pyarrow) - snappy [Standard]
4         3.18  Parquet (fastparquet) - snappy [Standard]
5         0.43               Parquet (pyarrow) - zstd [3]
6         0.40              Parquet (pyarrow) - zstd [10]
7         0.40              Parquet (pyarrow) - zstd [11]
8         3.54    Parquet (fastparquet) - zstd [Standard]
9         0.51             Parquet (pyarrow) - brotli [5]
10        0.53            Parquet (pyarrow) - brotli [10]
11        0.51            Parquet (pyarrow) - brotli [11]
12        3.34  Parquet (fastparquet) - brotli [Standard]
```

[6]:
```
display(df_results.describe().T.round(3))
```

```
               count     mean      std    min    25%    50%     75%      max
Größe_MB        13.0  120.298  188.984  26.91  49.69  50.98  101.77   736.48
Schreibzeit_s   13.0  164.295  369.519   2.85   6.61  10.56   90.78  1316.48
Lesezeit_s      13.0    5.199    8.141   0.40   0.47   0.53    3.54    27.37
```

1.2 Visuelle Auswertung und Framework-Vergleich

Zur besseren Vergleichbarkeit werden die erfassten Metriken nachfolgend grafisch aufbereitet.

Um das optimale Tooling für den finalen Bericht zu evaluieren, werden die Visualisierungen parallel mit drei gängigen Bibliotheken erstellt: **Seaborn**, **Plotly Express** und **Altair**.

1.2.1 seaborn

[7]:
```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

metrics = ["Größe_MB", "Schreibzeit_s", "Lesezeit_s"]

for metric in metrics:
    fig, ax = plt.subplots(figsize=(10, 6), constrained_layout=True)
```

```
    sns.barplot(
        data=df_results,
        x=metric,
        y="Variante",
        hue="Engine",
        palette="viridis",
        ax=ax,
    )
    ax.set_title(f"{metric} - Seaborn", fontsize=16)
    ax.set_xlabel(metric)
    ax.set_ylabel("Variante")

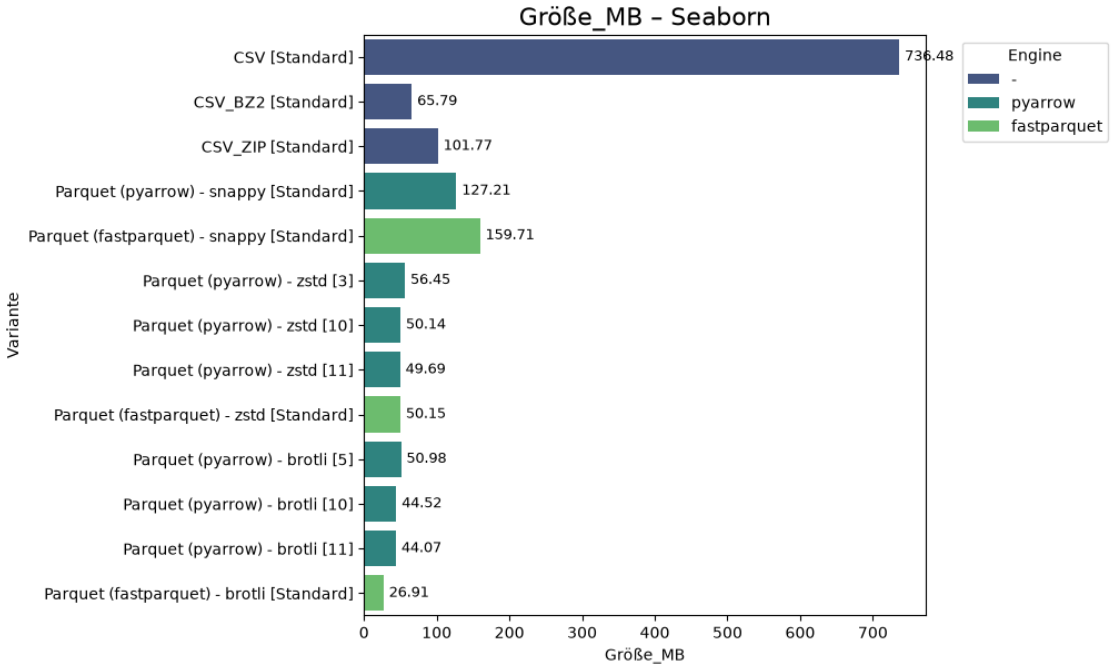
    # Maximalen Wert ermitteln, um den Textabstand dynamisch zu berechnen
    max_value = df_results[metric].max()
    # 1% des Maximalwerts als sauberer, skalenunabhängiger Abstand nach dem␣
↪Balken
    padding = max_value * 0.01

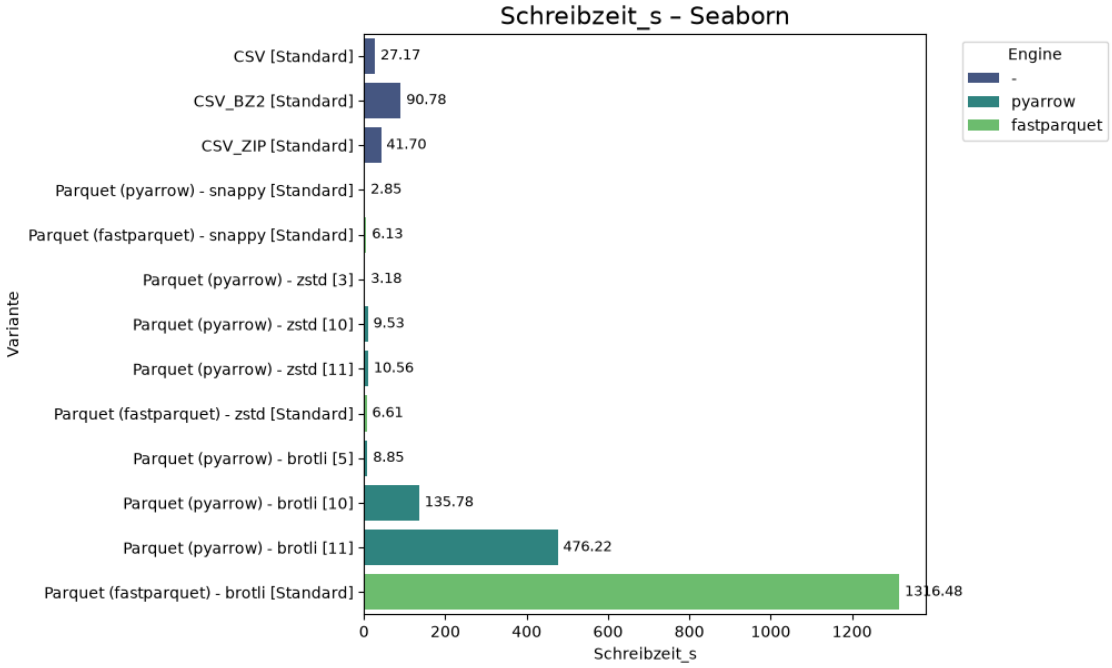
    # Werte an die echten Balken schreiben
    for p in ax.patches:
        width = p.get_width()

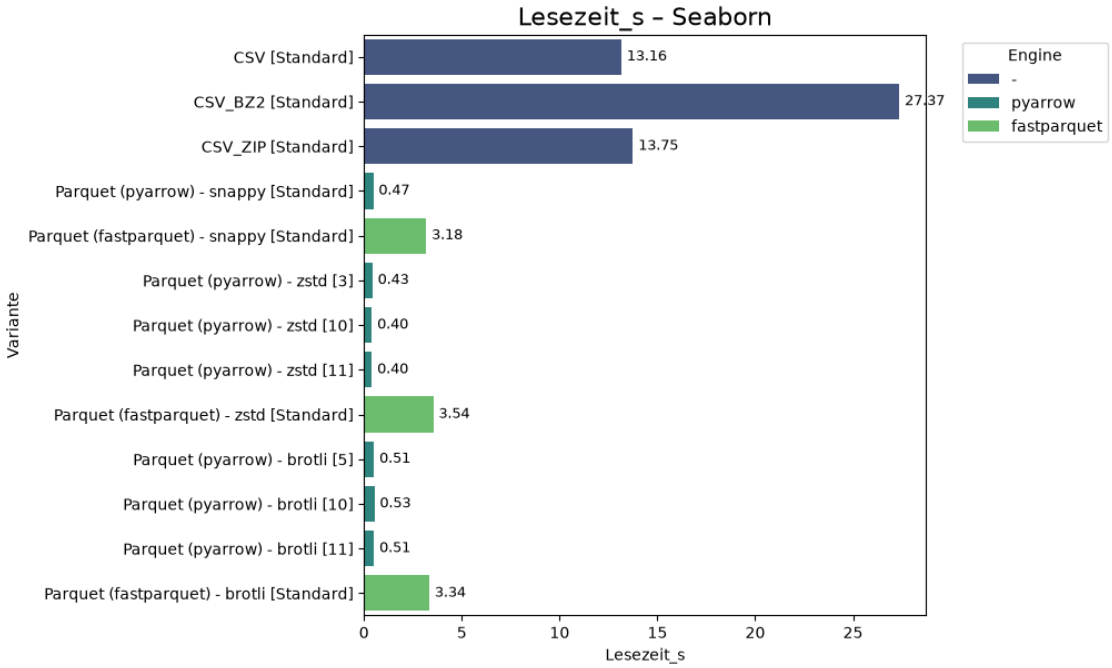
        # NUR beschriften, wenn der Balken existiert, nicht NaN ist und größer␣
↪als 0 ist
        if width > 0 and not np.isnan(width):
            # Berechnet die vertikale Mitte des aktuellen Balkens
            y_pos = p.get_y() + p.get_height() / 2

            ax.text(
                width + padding,
                y_pos,
                f"{width:.2f}",
                va="center",
                ha="left",  # Richtet den Text linksbündig an der berechneten␣
↪Position aus
                fontsize=9,
            )

    # Legende außerhalb platzieren
    ax.legend(title="Engine", bbox_to_anchor=(1.05, 1), loc="upper left")
    plt.show()
```


Größe_MB – Seaborn
CSV [Standard]
CSV_BZ2 [Standard]
CSV_ZIP [Standard]
Parquet (pyarrow) - snappy [Standard]
Parquet (fastparquet) - snappy [Standard]
Parquet (pyarrow) - zstd [3]
Parquet (pyarrow) - zstd [10]
Parquet (pyarrow) - zstd [11]
Parquet (fastparquet) - zstd [Standard]
Parquet (pyarrow) - brotli [5]
Parquet (pyarrow) - brotli [10]
Parquet (pyarrow) - brotli [11]
Parquet (fastparquet) - brotli [Standard]
736.48
65.79
101.77
127.21
159.71
56.45
50.14
49.69
50.15
50.98
44.52
44.07
26.91
Variante
0
100
200
300
400
500
600
700
Größe_MB
Engine
-
pyarrow
fastparquet

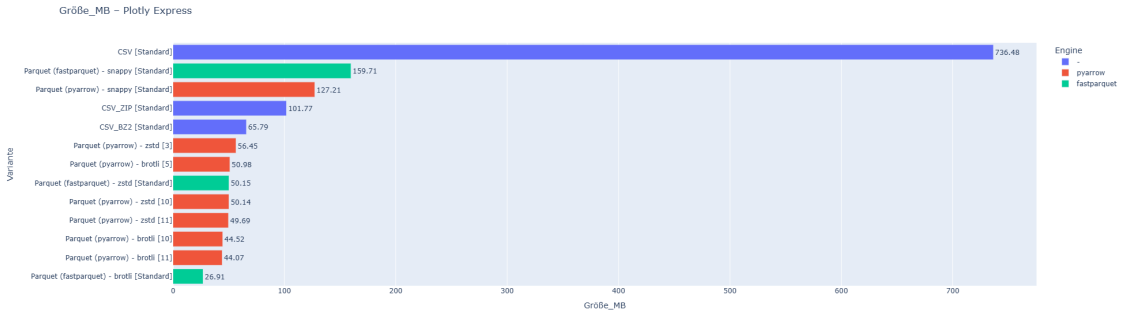

Schreibzeit_s – Seaborn
CSV [Standard]
CSV_BZ2 [Standard]
CSV_ZIP [Standard]
Parquet (pyarrow) - snappy [Standard]
Parquet (fastparquet) - snappy [Standard]
Parquet (pyarrow) - zstd [3]
Parquet (pyarrow) - zstd [10]
Parquet (pyarrow) - zstd [11]
Parquet (fastparquet) - zstd [Standard]
Parquet (pyarrow) - brotli [5]
Parquet (pyarrow) - brotli [10]
Parquet (pyarrow) - brotli [11]
Parquet (fastparquet) - brotli [Standard]
27.17
90.78
41.70
2.85
6.13
3.18
9.53
10.56
6.61
8.85
135.78
476.22
1316.48
Variante
0
200
400
600
800
1000
1200
Schreibzeit_s
Engine
-
pyarrow
fastparquet

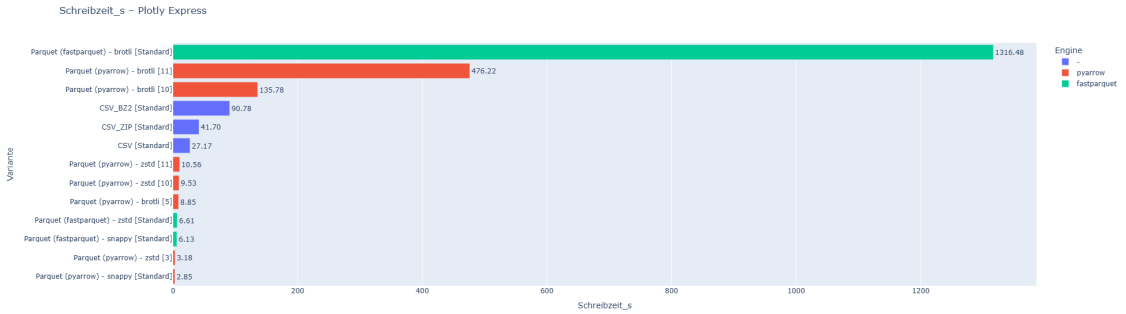


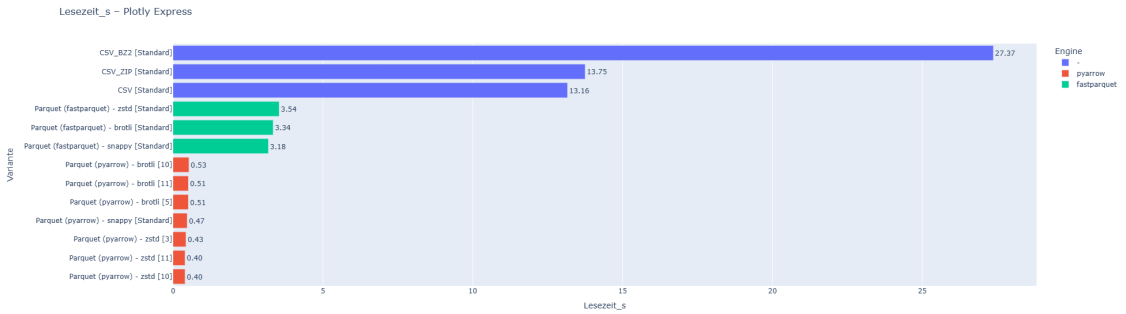
1.2.2 plotly express

[8]:
```
metrics = ["Größe_MB", "Schreibzeit_s", "Lesezeit_s"]

for metric in metrics:
    fig = px.bar(
        df_results,
        x=metric,
        y="Variante",
        color="Engine",
        text=metric,
        orientation="h",
        title=f"{metric} - Plotly Express",
        height=600,
        width=900
    )
    fig.update_traces(texttemplate='%{text:.2f}', textposition='outside')
    fig.update_layout(
        legend_title_text="Engine",
        yaxis=dict(categoryorder="total ascending")
    )
    fig.show()
```







1.2.3 altair

[9]:
```
metrics = ["Größe_MB", "Schreibzeit_s", "Lesezeit_s"]

for metric in metrics:
    chart = alt.Chart(df_results).mark_bar().encode(
        x=alt.X(metric, title=metric),
        y=alt.Y("Variante", sort="-x"),
        color="Engine",
```

```
        tooltip=["Format", "Engine", "Codec", "Level", metric]
    ).properties(
        title=f"{metric} - Altair",
        width=600,
        height=500
    )
    chart.display()
```

```
alt.Chart(…)
```

```
alt.Chart(…)
```

```
alt.Chart(…)
```

[10]:
```
from IPython.display import display, Markdown

# Kopie vorbereiten
df_copy = df_results.copy()

# Spaltennamen ggf. an deine tatsächlichen Namen anpassen
df_copy["Gesamtzeit_s"] = df_copy["Schreibzeit_s"] + df_copy["Lesezeit_s"]

# Hilfsfunktion: erste passende Zeile finden
def zeile(format_name=None, engine=None, codec=None, level=None):
    maske = df_copy["Format"].eq(format_name) if format_name else True

    if engine is not None:
        maske &= df_copy["Engine"].eq(engine)

    if codec is not None:
        maske &= df_copy["Codec"].eq(codec)

    if level is not None:
        maske &= df_copy["Level"].eq(level)

    ergebnis = df_copy[maske]

    if ergebnis.empty:
        return None

    return ergebnis.iloc[0]


def wert(row, spalte, nachkommastellen=2):
    if row is None:
        return "nicht vorhanden"
    return f"{row[spalte]:.{nachkommastellen}f}"
```

```
# Beispielhafte Datensätze auswählen
csv_standard = zeile(format_name="CSV", level="Standard")
csv_zip = zeile(format_name="CSV_ZIP")
csv_bz2 = zeile(format_name="CSV_BZ2", level="Standard")

parquet_pyarrow_snappy = zeile(
    format_name="Parquet",
    engine="pyarrow",
    codec="snappy"
)

parquet_fastparquet_snappy = zeile(
    format_name="Parquet",
    engine="fastparquet",
    codec="snappy"
)

parquet_fastparquet_brotli = zeile(
    format_name="Parquet",
    engine="fastparquet",
    codec="brotli",
    level="Standard"
)

parquet_pyarrow_zstd = zeile(
    format_name="Parquet",
    engine="pyarrow",
    codec="zstd",
    level=3
)
parquet_pyarrow_zstd10 = zeile(
    format_name="Parquet",
    engine="pyarrow",
    codec="zstd",
    level=10
)

parquet_pyarrow_zstd11 = zeile(
    format_name="Parquet",
    engine="pyarrow",
    codec="zstd",
    level=11
)

parquet_pyarrow_brotly10 = zeile(
    format_name="Parquet",
```

```
    engine="pyarrow",
    codec="brotli",
    level=10
)

parquet_pyarrow_brotly11 = zeile(
    format_name="Parquet",
    engine="pyarrow",
    codec="brotli",
    level=11
)

# Schnellste und kleinste Variante automatisch bestimmen
schnellste_schreibvariante = df_copy.loc[df_copy["Schreibzeit_s"].idxmin()]
schnellste_lesevariante = df_copy.loc[df_copy["Lesezeit_s"].idxmin()]
kleinste_datei = df_copy.loc[df_copy["Größe_MB"].idxmin()]

brotli_diff = (
    parquet_pyarrow_brotly11["Schreibzeit_s"]
    - parquet_pyarrow_brotly10["Schreibzeit_s"]
)

brotli_diff_str = f"{brotli_diff:2f}"
print(brotli_diff_str)
```

```
340.440000
```

[11]:
```
markdown_text = f"""
## Abschließende Evaluierung und Handlungsempfehlung

Die Durchführung des Benchmarks liefert eindeutige algorithmische Messwerte␣
 ↪bezüglich Dateigröße,
Schreibgeschwindigkeit und Lesezeit.
Basierend auf diesen Daten lassen sich differenzierte Kriterien für die Wahl␣
 ↪des optimalen Speicherformats ableiten.

### Automatische Gesamtbewertung

Die kleinste Datei erzeugt die Variante
**{kleinste_datei["Variante"]}** mit einer Größe von
**{kleinste_datei["Größe_MB"]:.2f} MB**.

Die schnellste Schreibzeit erreicht
**{schnellste_schreibvariante["Variante"]}**
mit **{schnellste_schreibvariante["Schreibzeit_s"]:.2f} Sekunden**.

Die schnellste Lesezeit erreicht
```

```
**{schnellste_lesevariante["Variante"]}**
mit **{schnellste_lesevariante["Lesezeit_s"]:.2f} Sekunden**.

Die gesamte Verarbeitung aus Schreiben und Lesen dauert bei der kleinsten
Datei **{kleinste_datei["Gesamtzeit_s"]:.2f} Sekunden**.


### Die Rolle klassischer Text- und Archivierungsformate (CSV, ZIP, BZ2)
Klassische textbasierte Formate bieten den fundamentalen Vorteil der␣
 ↪universellen Lesbarkeit,
weisen jedoch je nach Kompression deutliche Performance-Unterschiede auf:

* **Standard-CSV** ist die unkomprimierte CSV-Datei und der absolute␣
 ↪Spitzenreiter.
Der kritische Nachteil liegt jedoch im massiven Speicherbedarf von␣
 ↪**{wert(csv_standard, "Größe_MB")} MB**.
Die Schreibzeit beträgt **{wert(csv_standard, "Schreibzeit_s")} Sekunden**,
  die Lesezeit **{wert(csv_standard, "Lesezeit_s")} Sekunden**.
* **CSV_ZIP:** Die Kompression via ZIP bietet einen soliden Mittelweg.
Sie reduziert die Dateigröße effizient auf **{wert(csv_zip, "Größe_MB")} MB**␣
 ↪bei moderaten Verarbeitungszeiten
(Schreiben: **{wert(csv_zip, "Schreibzeit_s")} Sekunden** und Lesen:␣
 ↪**{wert(csv_zip, "Lesezeit_s")} Sekunden**).
* **CSV_BZ2 (Bzip2):** Dieses Format erzielt mit **{wert(csv_bz2, "Größe_MB")}␣
 ↪MB** eine der stärksten Kompressionen im Testfeld.
In der hier getesteten Standardkonfiguration von Pandas (maximale␣
 ↪Kompressionsstufe 9) führt dies jedoch zu einer extrem hohen CPU-Last,
was sich in einer Schreibzeit von **{wert(csv_bz2, "Schreibzeit_s")} Sekunden**␣
 ↪und einer Lesezeit von **{wert(csv_bz2, "Lesezeit_s")} Sekunden**␣
 ↪widerspiegelt.
  * *Hinweis zur Systemarchitektur:* In reinen Pandas-Umgebungen bremst BZ2 den␣
 ↪Workflow aus.
  In großen Big-Data-Infrastrukturen (wie Apache Spark oder Hadoop) ist BZ2␣
 ↪jedoch hochgradig relevant,
  da das Format im Gegensatz zu vielen anderen Codecs vollständig **splittbar**␣
 ↪ist.
  Große Datenmengen können somit trotz Kompression auf Cluster-Ebene parallel␣
 ↪von vielen Knoten gleichzeitig verarbeitet werden.
  Zudem könnte die Schreibzeit durch eine manuelle Herabsetzung der internen␣
 ↪Kompressionsstufe bei Bedarf optimiert werden.

### Der Technologiewechsel bei Binärformaten: PyArrow als Standard

Sobald das Projekt den Bereich der klassischen Textdateien verlässt und auf das␣
 ↪spaltenorientierte **Parquet-Format** setzt,
```

```
zeigt sich eine deutliche technologische Überlegenheit der **PyArrow-Engine**␣
 ↪gegenüber *fastparquet*:

* **Status von fastparquet:** Die Engine *fastparquet* gilt im modernen␣
 ↪Data-Science-Ökosystem (spätestens mit der Einführung von Pandas 3.0 und der␣
 ↪Standardisierung auf das Apache-Arrow-Backend) als weitgehend veraltet␣
 ↪(*deprecated*).
* **Direkter Performance-Vergleich:** Bei identischen Rahmenbedingungen (im␣
 ↪Standard-Snappy-Verfahren) übertrifft PyArrow die fastparquet-Engine in␣
 ↪allen Metriken deutlich:
  * **Dateigröße:** PyArrow komprimiert effizienter␣
 ↪(**{wert(parquet_pyarrow_snappy, "Größe_MB")} MB** vs.␣
 ↪**{wert(parquet_fastparquet_snappy, "Größe_MB")} MB**).
  * **Schreibzeit:** PyArrow arbeitet mehr als doppelt so schnell␣
 ↪(**{wert(parquet_pyarrow_snappy, "Schreibzeit_s")} Sekunden** vs.␣
 ↪**{wert(parquet_fastparquet_snappy, "Schreibzeit_s")} Sekunden**).
  * **Lesezeit:** PyArrow operiert im Millisekundenbereich␣
 ↪(**{wert(parquet_pyarrow_snappy, "Lesezeit_s")} Sekunden**), während␣
 ↪fastparquet mit **{wert(parquet_fastparquet_snappy, "Lesezeit_s")}␣
 ↪Sekunden** den Leseprozess spürbar verlangsamt.

### Kritische Relevanz von Codecs und Kompressionsstufen (Levels)

Die Analyse zeigt, dass eine Erhöhung der Kompressionsstufen ab einem␣
 ↪bestimmten Punkt in keinem Verhältnis mehr zur gewonnenen␣
 ↪Speicherplatzersparnis steht:

* **Das Brotli-Effizienzproblem:** Der Codec `brotli` zeigt bei Erhöhung von␣
 ↪Level 10 auf Level 11 eine dramatische Ineffizienz unter PyArrow.
Während sich die Dateigröße kaum spürbar verändert␣
 ↪(**{wert(parquet_pyarrow_brotly10, "Größe_MB")} MB** zu␣
 ↪**{wert(parquet_pyarrow_brotly11, "Größe_MB")} MB**),
explodiert die Schreibzeit von **{wert(parquet_pyarrow_brotly10,␣
 ↪"Schreibzeit_s")} Sekunden** auf **{wert(parquet_pyarrow_brotly11,␣
 ↪"Schreibzeit_s")} Sekunden**
(eine Erhöhung um ca. **{brotli_diff:.2f} Sekunden**).
* Bei der fastparquet-Implementierung von Brotli bricht das System sogar bei␣
 ↪**{parquet_fastparquet_brotli["Variante"]}**
mit **{wert(parquet_fastparquet_brotli, "Schreibzeit_s")} Sekunden** komplett␣
 ↪ein.
* Nach dem Ausschluss von `brotli` verbleiben für den praktischen Einsatz bei␣
 ↪Parquet-Dateien zwei hocheffiziente Codecs unter PyArrow:
`zstd` und `snappy`. Die Wahl ist vom jeweiligen Anwendungsfall (lokal, Server,␣
 ↪Framework-Versionen) abhängig.

#### {parquet_pyarrow_zstd["Variante"]}
```

```
- Größe: {wert(parquet_pyarrow_zstd, "Größe_MB")} MB
- Schreibzeit: {wert(parquet_pyarrow_zstd, "Schreibzeit_s")} Sekunden
- Lesezeit: **{wert(parquet_pyarrow_zstd, "Lesezeit_s")}** Sekunden
- **Der Speicherplatz-Gewinner:** Exzellente Balance. Extrem kleine Datei bei␣
 ↪rasanten Lesezeiten.

#### {parquet_pyarrow_snappy["Variante"]}

- Größe: {wert(parquet_pyarrow_snappy, "Größe_MB")} MB
- Schreibzeit: **{wert(parquet_pyarrow_snappy, "Schreibzeit_s")}** Sekunden
- Lesezeit: {wert(parquet_pyarrow_snappy, "Lesezeit_s")} Sekunden
- **Der Kompatibilitäts-Gewinner:** Schnell beim Schreiben, universell␣
 ↪unterstützt.

**Erkenntnis:** Höhere Levels erzwingen rechenintensive Suchalgorithmen, die im␣
 ↪Data-Engineering-Alltag immense zeitliche Ressourcen verschwenden.

## Abwägung und finale Entscheidungshilfe

Für die finale Formatwahl im Projekt wird eine zweistufige Entscheidungsmatrix␣
 ↪empfohlen:

1. **Einsatz klassischer Formate:** Sofern eine direkte menschliche Lesbarkeit␣
 ↪oder die native Weiterverarbeitung in Standard-Bürosoftware (wie Microsoft␣
 ↪Excel)
zwingend erforderlich ist, bleibt das **Standard-CSV-Format** für schnelle␣
 ↪Zwischenschritte oder die **CSV-ZIP-Variante** für speicherschonende␣
 ↪Archivierung die primäre Wahl.

2. **Einsatz von Spezialformaten (Parquet):** Falls für spezielle␣
 ↪datentechnische Zwecke nicht mit klassischen Textformaten gearbeitet wird␣
 ↪und die Datenanalysen auf maximalen
Durchsatz optimiert werden sollen, ist der Einsatz von **Parquet unter der␣
 ↪PyArrow-Engine** zwingend angeraten. Innerhalb dieses Ökosystems existieren␣
 ↪zwei Optionen:
   * **Der technische Favorit (ZSTD):** Aus reiner Performance-Sicht bietet␣
 ↪**ZSTD (z. B. auf Level 3)** die beste Balance.
   Es reduziert den Speicherbedarf im Vergleich zu Snappy um mehr als die␣
 ↪Hälfte (**{wert(parquet_pyarrow_zstd, "Größe_MB")} MB**),
   hält die Lesezeit bei rasanten **{wert(parquet_pyarrow_zstd, "Lesezeit_s")}␣
 ↪Sekunden** und wird von modernen Frameworks (Polars, DuckDB, moderne␣
 ↪Pandas-Versionen) nativ unterstützt.
   * **Die kompatible Projekt-Empfehlung (Snappy):** Trotz des höheren␣
 ↪Speicherbedarfs (**{wert(parquet_pyarrow_snappy, "Größe_MB")} MB**) wird
```

```
   für das vorliegende Projekt die Verwendung von **Snappy (Standard)**␣
 ↪präferiert. Der Grund hierfür liegt in der **maximalen␣
 ↪Abwärtskompatibilität**.
   Ältere Cloud-Infrastrukturen, Legacy-Hadoop-Systeme oder ältere␣
 ↪Server-Bibliotheken setzen Snappy oft zwingend voraus oder nutzen es als␣
 ↪festen Standard-Codec.

**Fazit:** Wenn Speicherplatz auf Systemen die primäre Engstelle darstellt, ist␣
 ↪der Wechsel auf `pyarrow` mit `zstd` (Level 3) dringend empfohlen.
Steht jedoch die reibungslose, fehlerfreie Integration in bestehende,␣
 ↪potenziell ältere Pipelines und gemischte Framework-Umgebungen im␣
 ↪Vordergrund,
bleibt `pyarrow` mit `snappy` der sicherste und kompatibelste Standard für␣
 ↪dieses Projekt.
"""

display(Markdown(markdown_text))
```

1.3 Abschließende Evaluierung und Handlungsempfehlung

Die Durchführung des Benchmarks liefert eindeutige algorithmische Messwerte bezüglich Dateigröße, Schreibgeschwindigkeit und Lesezeit. Basierend auf diesen Daten lassen sich differenzierte Kriterien für die Wahl des optimalen Speicherformats ableiten.

1.3.1 Automatische Gesamtbewertung

Die kleinste Datei erzeugt die Variante **Parquet (fastparquet) - brotli [Standard]** mit einer Größe von **26.91 MB**.

Die schnellste Schreibzeit erreicht **Parquet (pyarrow) - snappy [Standard]** mit **2.85 Sekunden**.

Die schnellste Lesezeit erreicht **Parquet (pyarrow) - zstd [10]** mit **0.40 Sekunden**.

Die gesamte Verarbeitung aus Schreiben und Lesen dauert bei der kleinsten Datei **1319.82 Sekunden**.

1.3.2 Die Rolle klassischer Text- und Archivierungsformate (CSV, ZIP, BZ2)

Klassische textbasierte Formate bieten den fundamentalen Vorteil der universellen Lesbarkeit, weisen jedoch je nach Kompression deutliche Performance-Unterschiede auf:

- **Standard-CSV** ist die unkomprimierte CSV-Datei und der absolute Spitzenreiter. Der kritische Nachteil liegt jedoch im massiven Speicherbedarf von **736.48 MB**. Die Schreibzeit beträgt **27.17 Sekunden**, die Lesezeit **13.16 Sekunden**.
- **CSV_ZIP:** Die Kompression via ZIP bietet einen soliden Mittelweg. Sie reduziert die Dateigröße effizient auf **101.77 MB** bei moderaten Verarbeitungszeiten (Schreiben: **41.70 Sekunden** und Lesen: **13.75 Sekunden**).
- **CSV_BZ2 (Bzip2):** Dieses Format erzielt mit **65.79 MB** eine der stärksten Kompressionen im Testfeld. In der hier getesteten Standardkonfiguration von Pandas (maximale Kompres-

sionsstufe 9) führt dies jedoch zu einer extrem hohen CPU-Last, was sich in einer Schreibzeit von **90.78 Sekunden** und einer Lesezeit von **27.37 Sekunden** widerspiegelt.
  - *Hinweis zur Systemarchitektur:* In reinen Pandas-Umgebungen bremst BZ2 den Workflow aus. In großen Big-Data-Infrastrukturen (wie Apache Spark oder Hadoop) ist BZ2 jedoch hochgradig relevant, da das Format im Gegensatz zu vielen anderen Codecs vollständig **splittbar** ist. Große Datenmengen können somit trotz Kompression auf Cluster-Ebene parallel von vielen Knoten gleichzeitig verarbeitet werden. Zudem könnte die Schreibzeit durch eine manuelle Herabsetzung der internen Kompressionsstufe bei Bedarf optimiert werden.

1.3.3 Der Technologiewechsel bei Binärformaten: PyArrow als Standard

Sobald das Projekt den Bereich der klassischen Textdateien verlässt und auf das spaltenorientierte **Parquet-Format** setzt, zeigt sich eine deutliche technologische Überlegenheit der **PyArrow-Engine** gegenüber *fastparquet*:

- **Status von fastparquet:** Die Engine *fastparquet* gilt im modernen Data-Science-Ökosystem (spätestens mit der Einführung von Pandas 3.0 und der Standardisierung auf das Apache-Arrow-Backend) als weitgehend veraltet (*deprecated*).
- **Direkter Performance-Vergleich:** Bei identischen Rahmenbedingungen (im Standard-Snappy-Verfahren) übertrifft PyArrow die fastparquet-Engine in allen Metriken deutlich:
  - **Dateigröße:** PyArrow komprimiert effizienter (**127.21 MB** vs. **159.71 MB**).
  - **Schreibzeit:** PyArrow arbeitet mehr als doppelt so schnell (**2.85 Sekunden** vs. **6.13 Sekunden**).
  - **Lesezeit:** PyArrow operiert im Millisekundenbereich (**0.47 Sekunden**), während fastparquet mit **3.18 Sekunden** den Leseprozess spürbar verlangsamt.

1.3.4 Kritische Relevanz von Codecs und Kompressionsstufen (Levels)

Die Analyse zeigt, dass eine Erhöhung der Kompressionsstufen ab einem bestimmten Punkt in keinem Verhältnis mehr zur gewonnenen Speicherplatzersparnis steht:

- **Das Brotli-Effizienzproblem:** Der Codec `brotli` zeigt bei Erhöhung von Level 10 auf Level 11 eine dramatische Ineffizienz unter PyArrow. Während sich die Dateigröße kaum spürbar verändert (**44.52 MB** zu **44.07 MB**), explodiert die Schreibzeit von **135.78 Sekunden** auf **476.22 Sekunden** (eine Erhöhung um ca. **340.44 Sekunden**).
- Bei der fastparquet-Implementierung von Brotli bricht das System sogar bei **Parquet (fastparquet) - brotli [Standard]** mit **1316.48 Sekunden** komplett ein.
- Nach dem Ausschluss von `brotli` verbleiben für den praktischen Einsatz bei Parquet-Dateien zwei hocheffiziente Codecs unter PyArrow: `zstd` und `snappy`. Die Wahl ist vom jeweiligen Anwendungsfall (lokal, Server, Framework-Versionen) abhängig.

Parquet (pyarrow) - zstd [3]

- Größe: 56.45 MB
- Schreibzeit: 3.18 Sekunden
- Lesezeit: **0.43** Sekunden
- **Der Speicherplatz-Gewinner:** Exzellente Balance. Extrem kleine Datei bei rasanten Lesezeiten.

Parquet (pyarrow) - snappy [Standard]

- Größe: 127.21 MB
- Schreibzeit: **2.85** Sekunden
- Lesezeit: 0.47 Sekunden
- **Der Kompatibilitäts-Gewinner:** Schnell beim Schreiben, universell unterstützt.

Erkenntnis: Höhere Levels erzwingen rechenintensive Suchalgorithmen, die im Data-Engineering-Alltag immense zeitliche Ressourcen verschwenden.

1.4 Abwägung und finale Entscheidungshilfe

Für die finale Formatwahl im Projekt wird eine zweistufige Entscheidungsmatrix empfohlen:

1. **Einsatz klassischer Formate:** Sofern eine direkte menschliche Lesbarkeit oder die native Weiterverarbeitung in Standard-Bürosoftware (wie Microsoft Excel) zwingend erforderlich ist, bleibt das **Standard-CSV-Format** für schnelle Zwischenschritte oder die **CSV-ZIP-Variante** für speicherschonende Archivierung die primäre Wahl.
2. **Einsatz von Spezialformaten (Parquet):** Falls für spezielle datentechnische Zwecke nicht mit klassischen Textformaten gearbeitet wird und die Datenanalysen auf maximalen Durchsatz optimiert werden sollen, ist der Einsatz von **Parquet unter der PyArrow-Engine** zwingend angeraten. Innerhalb dieses Ökosystems existieren zwei Optionen:
   - **Der technische Favorit (ZSTD):** Aus reiner Performance-Sicht bietet **ZSTD (z. B. auf Level 3)** die beste Balance. Es reduziert den Speicherbedarf im Vergleich zu Snappy um mehr als die Hälfte (**56.45 MB**), hält die Lesezeit bei rasanten **0.43 Sekunden** und wird von modernen Frameworks (Polars, DuckDB, moderne Pandas-Versionen) nativ unterstützt.
   - **Die kompatible Projekt-Empfehlung (Snappy):** Trotz des höheren Speicherbedarfs (**127.21 MB**) wird für das vorliegende Projekt die Verwendung von **Snappy (Standard)** präferiert. Der Grund hierfür liegt in der **maximalen Abwärtskompatibilität**. Ältere Cloud-Infrastrukturen, Legacy-Hadoop-Systeme oder ältere Server-Bibliotheken setzen Snappy oft zwingend voraus oder nutzen es als festen Standard-Codec.

Fazit: Wenn Speicherplatz auf Systemen die primäre Engstelle darstellt, ist der Wechsel auf `pyarrow` mit `zstd` (Level 3) dringend empfohlen. Steht jedoch die reibungslose, fehlerfreie Integration in bestehende, potenziell ältere Pipelines und gemischte Framework-Umgebungen im Vordergrund, bleibt `pyarrow` mit `snappy` der sicherste und kompatibelste Standard für dieses Projekt.

[12]:
```
# Level als String für Standart Werte
df_results_save = df_results.copy()

df_results_save["Level"] = (
    df_results_save["Level"]
    .astype("string")
)
# Zielordner sicher erstellen
DATEIFORMATE_RESULTS.parent.mkdir(
    parents=True,
```

```
    exist_ok=True,
)

# Finale Ergebnistabelle speichern
df_results_save.to_parquet(
    DATEIFORMATE_RESULTS,
    engine="pyarrow",
    compression="snappy",
    index=False,
)

print(f"Ergebnisse gespeichert unter:")
print(DATEIFORMATE_RESULTS)
print(f"Dateigröße: {DATEIFORMATE_RESULTS.stat().st_size / 1024**2:.2f} MB")
```

```
Ergebnisse gespeichert unter:
C:\0_DA\Source\Projekt_Deutsche_Bahn\outputs\01_results.parquet
Dateigröße: 0.01 MB
```

[ ]: